

Day 15 — intro to machine learning

MD Mutasim Billah · 52-week Data Science to ML/AI roadmap · Week 3, Day 1 — paired with `day15_intro_ml.py`

1. From describing data to predicting it

Weeks 1-2 answered "what happened": who survived, what correlates with what, how to prove it in SQL. Machine learning asks a different question — given a new passenger never seen before, can a model predict whether they'd survive? Same Titanic data, new goal.

2. Supervised learning, features, and target

Titanic has 891 rows where the answer is already known (Survived). Supervised learning means: show the model many examples with known answers, let it find the pattern, then ask it to guess the answer for new examples. Every supervised problem starts by splitting columns into two groups.

```
y = df["Survived"]

feature_cols = ["Pclass", "Sex", "Age", "SibSp", "Parch", "Fare",
               "Embarked", "FamilySize"]
X = df[feature_cols].copy()
```

- `y` (target): the column being predicted
- `X` (features): the columns allowed to be used to predict it
- `PassengerId`, `Name`, and `Ticket` are dropped — they're identifiers, not predictive signal

3. Encoding: models only understand numbers

`Sex` and `Embarked` are text categories. A model can't do math on the word "female" — it needs to become a number first.

```
X_encoded = pd.get_dummies(X, columns=["Sex", "Embarked"], drop_first=True)
```

- `get_dummies` turns one category column into several 0/1 columns, one per category
- `drop_first=True` drops one category to avoid redundancy — if a row isn't male, isn't Embarked_Q, and isn't Embarked_S, that already implies female and Embarked_C, so a fourth column would just repeat information the other three already carry

4. Train/test split: never grade your own homework

This is the single most important habit in machine learning. If a model is evaluated on the same data it learned from, it can score well simply by memorizing answers — that tells you nothing about how it handles a new passenger.

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
```

- 80% of rows train the model, the remaining 20% are held back and never shown to it until evaluation time
- `random_state=42` makes the random split reproducible — same split every time the script runs
- `stratify=y` keeps the survival ratio consistent in both the train and test slices, so the test set isn't accidentally easier or harder than the training set

5. Training the baseline model

Logistic regression is the standard first model for a yes/no prediction problem like this one — despite the name, it's a classifier, not a number predictor.

```
model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)
predictions = model.predict(X_test)
```

- `.fit()` is where the learning happens — the model looks at `X_train` and `y_train` together and works out the relationship between them
- `.predict()` applies that learned relationship to new rows (`X_test`) it has never seen paired with an answer
- `max_iter=1000` raises the optimizer's iteration cap — the default sometimes isn't enough for the training process to fully converge

6. Reading the coefficients

Logistic regression assigns one coefficient to every feature — a signed number showing how much that feature pushes a prediction toward survival or death.

```
coefs = pd.Series(model.coef_[0], index=X_train.columns).sort_values()
```

A large negative coefficient on `Sex_male` (roughly -1.65 in a test run) means being male strongly pushed predictions toward "died" — matching exactly what Day 10's chi-square test already found, now discovered independently by the model itself.

7. Evaluation: why accuracy alone is misleading

```
accuracy_score(y_test, predictions)
confusion_matrix(y_test, predictions)
classification_report(y_test, predictions, target_names=["Died", "Survived"])
```

- Accuracy is just percent correct overall — it can look good even when a model completely fails one class, if that class is rare
- A confusion matrix breaks predictions into 4 buckets: correctly predicted deaths, correctly predicted survivals, and the two kinds of mistakes
- The script also computes a naive baseline (always predicting "died") — a real model needs to beat that naive guess by a meaningful margin, not just score a high-looking accuracy number in isolation

8. Deliverable and push

```
git add day15_intro_ml.py
git commit -m "Day 15: Intro to ML - train/test split, logistic regression baseline, e
git push
```